

Hima Automation Tests

Transactions module — dual-account coverage · 45 automated checks



PROJECT	Hima · Android	TARGET SCREENS	Male + Female Transactions
ENDPOINTS	/transaction_list /female_transaction	ENVIRONMENT	https://demohima.himaapp.in/api/auth
TEST ACCOUNTS	Male id 456976 · Female id 456740	SUITE TYPE	Read-only API automation
AUTOMATED BY	transactions_api_tests.py	QA LEAD	Perumal

Report generated: 2026-04-23 14:54 · Suite version: v2 (dual-account)

01 Executive summary

For the tech lead — one paragraph

Automated API tests ran 45 checks against the two transaction endpoints with both a male and a female logged-in account. **35** passed and **10** failed. Failures map to **6 distinct bugs**, **100%** fixable with code we own. One is **CRITICAL**: `/transaction_list` exposes any user's financial history to any authenticated caller. The sibling `/female_transaction` endpoint already enforces the right check — the fix is a copy-paste.

Coverage spans functional, pagination, boundary, negative, security (IDOR, auth, SQLi, gender-gate, cross-endpoint), HTTP-verb correctness, response-contract and performance p95 categories. The female endpoint is the hardened reference — nearly every failure is on the male endpoint or shared across both.

02 Who needs to ship what

Every failure is classified by where the fix closes the hole. A fix belongs to the backend if a curl-wielding attacker bypasses any client-side guard.

Android only

0 / 10

- No bug is purely client-fixable in this suite.

Android + Backend

3 / 10

- `TransactionsResponse` schema (client nullable ✓ · server should emit keys)
- HTML 500 on `limit=-1` — client error observer ✓ · server validator needed

Backend only

7 / 10

- **CRITICAL**: IDOR on `/transaction_list` — bind `user_id` to JWT sub
- Cap `'limit'` at 100 on both endpoints
- Reject negative offset with 400 JSON
- Force JSON 401 on `/api/*` when unauthenticated

Bottom line: 100% of the 10 failing cases (10 of 10) are addressable with code we control. Zero need new infrastructure.

03 Bugs — plain-language breakdown

Ordered critical first. Each entry lists what's wrong, why it hurts, and how to fix.

BUG-TX-001 CRITICAL FIX: SERVER MALE TRANSACTIONS

/transaction_list — IDOR: male token reads FEMALE user's txns

What's wrong	Any logged-in user can read any other user's complete transaction history (amount, payment_type, datetime) by sending a different user_id in the request body on /transaction_list.
Why it matters	CRITICAL. Financial-history privacy breach across all users. Test proof: male QA token (id 456976) fetched 272 rows belonging to female QA account (id 456740) on demo — same pattern confirmed on prod (8,926 rows of user 72530). Reportable under DPDP / GDPR.
How to fix	Backend: derive user_id from the JWT sub claim, ignore the body value. The sibling /female_transaction already does this — copy that pattern.
Source reference	app/src/main/java/com/gmwapp/hima/retrofit/ApiManager.kt:1848–1858
Server response	<pre>{"success":true,"message":"User transaction list retrieved successfully.,"total":272,"data":[{"id":7093224,"user_id":456740,"type":"cancelled","amount":"10.00","payment_type":"","datetime":"2026-03-31 20:34:42","date":"Mar 31","coins":0},{id":7093217,"user_i</pre>

BUG-TX-004 HIGH FIX: BOTH FEMALE TRANSACTIONS

/female_transaction — negative limit (HTML 500 leak on female too?)

What's wrong	Sending limit=-1 to either endpoint returns HTTP 500 with the Laravel HTML error page instead of a JSON error. Confirmed on BOTH /transaction_list and /female_transaction.
Why it matters	High. Breaks the Android client (it can't parse HTML as JSON → blank screen with no feedback). Information-disclosure risk if APP_DEBUG=true on prod.
How to fix	Android: DONE — TransactionsActivity now observes errorLiveData and shows a toast. FemaleTransactionsActivity already had this. Backend: add 'limit' => 'integer min:1 max:100' validator — one change protects both endpoints.
Server response	<pre>{ "message": "Server Error" }</pre>

BUG-TX-005 HIGH FIX: SERVER FEMALE TRANSACTIONS

/female_transaction — limit=99999 also uncapped?

What's wrong	Neither endpoint caps the 'limit' parameter. limit=99999 returns every row in one response — 272 on demo, 8,926 on prod.
Why it matters	High. Two attack surfaces: DoS (a few thousand requests exhaust MySQL connections + bandwidth) and mobile OOM (Gson blows up parsing on older devices). Confirmed on BOTH endpoints.
How to fix	Backend: add 'limit' => 'nullable integer min:1 max:100' validator and enforce the cap server-side. One validator protects both endpoints.
Server response	<pre>{"success":true,"message":"Female transaction list retrieved successfully.,"total":262,"data":[{"id":7093032,"user_id":456740,"type":"audio","amount":0,"coins":0,"payment_type":null,"datetime":"2026-03-26 16:07:50","date":"Mar 26","title":"Audio session with</pre>

BUG-TX-007

HIGH

FIX: BOTH

MALE TRANSACTIONS

/transaction_list — response schema matches TransactionsResponse.kt

What's wrong	TransactionsResponse.kt originally declared user_name, call_type, duration, call_user_name as non-nullable Kotlin Strings, but the server never sends them. Confirmed by TXM-02 — 5 rows inspected, all 4 fields missing.
Why it matters	High (latent). Today Gson silently defaults to null, violating Kotlin's null contract. On a Kotlin 1.9+ upgrade with strict null safety the Transactions screen crashes on first deserialize.
How to fix	Android: DONE — fields are now nullable in the data class, adapter code handles null via .orEmpty(). Backend: still should emit the documented keys (empty string fallback) for contract stability.
Source reference	app/src/main/java/com/gmwapp/hima/retrofit/responses/TransactionsResponse.kt:9-23
Server response	<pre>{"success":true,"message":"User transaction list retrieved successfully.","total":272,"data":[{"id":7093224,"user_id":456740,"type":"cancelled","amount":"10.00","payment_type":"","datetime":"2026-03-31 20:34:42","date":"Mar 31","coins":0},{id":7093217,"user_i</pre>

BUG-TX-003

MEDIUM

FIX: SERVER

FEMALE TRANSACTIONS

/female_transaction — negative offset (silent coercion on female too?)

What's wrong	Negative offset values (offset=-1) are silently coerced to 0 on BOTH /transaction_list AND /female_transaction instead of being rejected.
Why it matters	Medium. A client with off-by-one pagination math can double-render rows without detection. Confirmed on both endpoints: first_id with offset=-1 equals first_id with offset=0.
How to fix	Backend: add 'offset' => 'nullable integer min:0' validator on both endpoints.
Server response	<pre>{"neg_first": 7093032, "zero_first": 7093032, "neg_success": true}</pre>

BUG-TX-002

MEDIUM

FIX: SERVER

MALE TRANSACTIONS

/transaction_list — no-auth without Accept:json returns HTML 302

What's wrong	When a client hits /transaction_list without Accept:application/json, the server responds with HTML 302 → /login instead of a JSON 401.
Why it matters	Medium. Retrofit sets Accept:json so the Android app is fine today, but any third-party client or misconfigured interceptor crashes its JSON parser against the HTML response.
How to fix	Backend: force JSON 401 for request()->is('api/*') in Laravel's exception handler.
Server response	<pre><!DOCTYPE html> <html> <head> <meta charset="UTF-8" /> <meta http-equiv="refresh" content="0;url='https://demohima.himaapp.in/login'" /> <title>Redirecting to https://demohima.himaapp.in/login</title> </head> <body></pre>

04 Full test matrix

All 45 cases grouped by screen. Failing rows highlighted.

Male Transactions (19 tests · 13 pass · 6 fail)

ID	CATEGORY	TEST	SEV	STATUS	HTTP	DETAIL
TXM-01	Functional	/transaction_list — male happy path (self, offset=0, limit=10)	CRITICAL	PASS	200	success=False msg='No transactions found for this user.'
TXM-02	Contract	/transaction_list — response schema matches TransactionsResponse.kt	HIGH	FAIL	200	row missing: ['call_type', 'call_user_name', 'duration', 'user_name']
TXM-03	Functional	/transaction_list — default pagination: limit=10 returns ≤10	MEDIUM	PASS	200	len=10
TXM-04	Functional	/transaction_list — offset advances correctly (page 2 ≠ page 1)	MEDIUM	PASS	200	ids differ {"page1_first": 7093224, "page2_first": 7093179}
TXM-05	Security	/transaction_list — IDOR: male token reads FEMALE user's txns	CRITICAL	FAIL	200	LEAKED 5 txns of user 456740 to male user 456976
TXM-06	Security	/transaction_list — no Authorization header (Accept: application/json)	HIGH	PASS	401	HTTP 401
TXM-06b	Contract	/transaction_list — no-auth without Accept:json returns HTML 302	MEDIUM	FAIL	302	HTTP 302 body starts with '<!DOCTYPE html>\n<html>\n <head>\n <meta charset="UTF-8" />\n <meta'
TXM-07	Security	/transaction_list — tampered/invalid Bearer token	HIGH	PASS	401	HTTP 401
TXM-08	Validation	/transaction_list — non-numeric user_id	MEDIUM	PASS	200	'Invalid user_id. Must be a numeric value.'
TXM-09	Security	/transaction_list — SQLi probe in user_id	HIGH	PASS	200	rejected early (Invalid user_id)
TXM-10	Validation	/transaction_list — negative offset silently coerced to 0?	MEDIUM	FAIL	200	silent coercion -1==0: {"neg_first": 7093224, "zero_first": 7093224, "neg_success": true}
TXM-11	Validation	/transaction_list — negative limit	HIGH	FAIL	500	HTTP 500 — { "message": "Server Error" }
TXM-12	Performance	/transaction_list — limit=99999 bypasses pagination → DoS	HIGH	FAIL	200	returned 272 rows in 0.25s — no 100-row cap
TXM-13	Validation	/transaction_list — missing user_id	MEDIUM	PASS	200	'user_id is required.'
TXM-14	Functional	/transaction_list — missing offset and limit (defaults)	INFO	PASS	200	len=10
TXM-15	Contract	/transaction_list — GET verb returns 405 (not 302)	MEDIUM	PASS	405	HTTP 405
TXM-16	Performance	/transaction_list — p95 budget < 2.0s for limit=10	MEDIUM	PASS	200	0.29s
TXM-17	Functional	/transaction_list — identical request twice is idempotent	INFO	PASS	200	{"t1": 272, "t2": 272}
TXM-18	Boundary	/transaction_list — offset beyond dataset returns empty / success=false	INFO	PASS	200	len=0

Female Transactions (17 tests · 14 pass · 3 fail)

ID	CATEGORY	TEST	SEV	STATUS	HTTP	DETAIL
TXF-01	Functional	/female_transaction — female happy path (self, offset=0, limit=10)	CRITICAL	PASS	200	total=262 len=10
TXF-02	Contract	/female_transaction — schema matches FemaleTransactionsResponse.kt	HIGH	PASS	200	10 rows, all 10 fields present
TXF-03	Functional	/female_transaction — default pagination: limit=10 returns ≤10	MEDIUM	PASS	200	len=10
TXF-04	Functional	/female_transaction — offset advances correctly (page 2 ≠ page 1)	MEDIUM	PASS	200	ids differ {"p1": 7093032, "p2": 7092960}
TXF-05	Security	/female_transaction — no Authorization header	HIGH	PASS	401	HTTP 401
TXF-06	Validation	/female_transaction — missing user_id	MEDIUM	PASS	200	'user_id is required.'
TXF-07	Validation	/female_transaction — non-numeric user_id	MEDIUM	PASS	200	'Invalid user_id. Must be a positive integer.'
TXF-08	Security	/female_transaction — SQLi probe in user_id	HIGH	PASS	403	rejected early
TXF-09	Validation	/female_transaction — negative limit (HTML 500 leak on female too?)	HIGH	FAIL	500	HTTP 500 — HTML 500 leak!
TXF-10	Validation	/female_transaction — negative offset (silent coercion on female too?)	MEDIUM	FAIL	200	silent coercion -1==0 on female too: {"neg_first": 7093032, "zero_first": 7093032, "neg_success": true}
TXF-11	Performance	/female_transaction — limit=99999 also uncapped?	HIGH	FAIL	200	returned 262 rows — no 100-row cap on female endpoint either
TXF-12	Functional	/female_transaction — missing offset and limit (defaults)	INFO	PASS	200	len=10
TXF-13	Validation	/female_transaction — GET verb returns 405	INFO	PASS	405	HTTP 405
TXF-14	Performance	/female_transaction — p95 budget < 2.0s for limit=10	MEDIUM	PASS	200	0.23s
TXF-15	Security	/female_transaction — invalid/tampered Bearer token	HIGH	PASS	401	HTTP 401
TXF-16	Functional	/female_transaction — idempotent repeat	INFO	PASS	200	{"t1": 262, "t2": 262}
TXF-17	Boundary	/female_transaction — offset beyond dataset returns empty	INFO	PASS	200	len=0

Cross-Account (9 tests · 8 pass · 1 fail)

ID	CATEGORY	TEST	SEV	STATUS	HTTP	DETAIL
TXX-01	Security	/female_transaction — MALE token calling FEMALE user_id	CRITICAL	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-02	Security	/female_transaction — FEMALE token calling MALE user_id (reverse)	CRITICAL	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-03	Security	/transaction_list — FEMALE token reading MALE user's txns (reverse IDOR)	CRITICAL	PASS	200	refused
TXX-04	Security	/female_transaction — MALE token using own MALE id (gender gate)	HIGH	PASS	200	'This endpoint is only for female users.'
TXX-05	Security	/transaction_list — FEMALE token reading FEMALE's OWN id (no gender gate)	MEDIUM	PASS	200	total=272
TXX-06	Security	/transaction_list — MALE token + FEMALE id + limit=99999 (compound IDOR+DoS)	CRITICAL	FAIL	200	compound leak: 272 foreign rows in 0.19s
TXX-07	Security	/female_transaction — MALE token, FEMALE id, limit=99999 (ownership fires first)	HIGH	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-08	Contract	Cross-endpoint schema parity: /transaction_list keys ⊆ /female_transaction row shape?	MEDIUM	PASS	200	documented: {"male_ep_keys": ["amount", "coins", "date", "datetime", "id", "payment_type", "type", "user_id"], "fem
TXX-09	Security	Token swap: MALE JWT + FEMALE user_id on /transaction_list — does server log origin?	MEDIUM	PASS	200	see note — requires server log audit

05 Methodology

- Both accounts authenticated via `/login` with QA backdoor OTP `011011`.
- Male account: `9876543210` → id `456976` · Female account: `9120444444` → id `456740`.
- Read-only suite — no call, recharge, gift, profile change or transaction was created on any account.
- Cross-account (TXX-*) cases use the *other* account's id to probe ownership / gender enforcement.
- 150 ms inter-request delay to avoid throttle middleware.
- Contract checks validate that response rows contain every field the Kotlin data class declares.
- p95 performance budget: 2.0 s per request.
- Manual on-device verification of the Android patches (BUG-TX-004, BUG-TX-007) is performed by Perumal — not by the automation script.

06 Recommended rollout order

1. **TODAY:** Deploy the IDOR fix on `/transaction_list` (BUG-TX-001). One-line change, mirrors `/female_transaction`. Closes the critical financial-data leak.
2. **This sprint:** Add `FormRequest` validators for `user_id` / `offset` / `limit` (closes BUG-TX-003, BUG-TX-004, BUG-TX-005 on both endpoints). Ship the already-merged Kotlin nullable change for `TransactionsResponse` (BUG-TX-007) in the next app release.
3. **Next sprint:** Force JSON-only responses for `/api/*` routes (BUG-TX-002). Contract cleanup.